

Trạng thái	Đã xong
Bắt đầu vào lúc	Thứ Bảy, 13 tháng 7 2024, 8:31 AM
Kết thúc lúc	Thứ Sáu, 23 tháng 8 2024, 3:24 PM
Thời gian thực hiện	41 Các ngày 6 giờ
Điểm	6,00/6,00
Điểm	10,00 trên 10,00 (100%)



Câu hỏi 1

Đúng

Đạt điểm 1,00 trên 1,00

Implement methods **add**, **size** in template class **DLinkedList (which implements List ADT)** representing the doubly linked list with type T with the initialized frame. The description of each method is given in the code.

```
template <class T>
class DLinkedList {
public:
    class Node; // Forward declaration
protected:
    Node* head;
    Node* tail;
    int count;
public:
    DLinkedList();
    ~DLinkedList();
    void add(const T &e);
    void add(int index, const T &e);
    int size();
public:
    class Node
    {
    private:
        T data;
        Node *next;
        Node *previous;
        friend class DLinkedList<T>;

    public:
        Node()
        {
            this->previous = NULL;
            this->next = NULL;
        }

        Node(const T &data)
        {
            this->data = data;
            this->previous = NULL;
            this->next = NULL;
        }
    };
};
```

In this exercise, we have include <iostream>, <string>, <sstream> and using namespace std.

For example:

Test	Result
DLinkedList<int> list; int size = 10; for(int idx=0; idx < size; idx++){ list.add(idx); } cout << list.toString();	[0,1,2,3,4,5,6,7,8,9]
DLinkedList<int> list; int size = 10; for(int idx=0; idx < size; idx++){ list.add(0, idx); } cout << list.toString();	[9,8,7,6,5,4,3,2,1,0]

Answer: (penalty regime: 0, 0, 0, 5, 10 %)

Reset answer

```
1
2 template <class T>
3 void DLinkedList<T>::add(const T& e) {
4     /* Insert an element into the end of the list. */
5     Node * tmp= new Node (e);
6     if (this->count==0) head=tail=tmp;
7     else {
8         tail->next= tmp;
9         tmp->previous= tail;
10        tail=tmp;
11    }
12
13    this->count++;
14
15
16
17 }
18
19 template<class T>
20 void DLinkedList<T>::add(int index, const T& e) {
21     /* Insert an element into the list at given index. */
22     if (index<0||index>count) throw out_of_range("index out of range");
23     Node * tmp= new Node (e);
24
25     if (this->count==0) {
26         head=tail=tmp;
27     }
28
29     if (index ==0 &&this->count>0){
30         tmp->next=head;
31         head->previous=tmp;
32         head=tmp;
33     }
34     else if( index ==count){
35         tail->next= tmp;
36         tmp->previous= tail;
37         tail=tmp;
38     }
39
40     else {
41         Node* current= head;
42         for(int i=0;i<index -1;i++){
43             current=current->next;
44         }
45         tmp->previous=current;
46         tmp->next=current->next;
47         current->next->previous=tmp;
48         current->next=tmp;
49     }
50
51     count++;
52
53 }
54
55
56 template<class T>
57 int DLinkedList<T>::size() {
58     /* Return the length (size) of list */
59     return this->count;
60 }
61
```

	Test	Expected	Got	
✓	<pre>DLinkedList<int> list; int size = 10; for(int idx=0; idx < size; idx++){ list.add(idx); } cout << list.toString();</pre>	[0,1,2,3,4,5,6,7,8,9]	[0,1,2,3,4,5,6,7,8,9]	✓
✓	<pre>DLinkedList<int> list; int size = 10; for(int idx=0; idx < size; idx++){ list.add(0, idx); } cout << list.toString();</pre>	[9,8,7,6,5,4,3,2,1,0]	[9,8,7,6,5,4,3,2,1,0]	✓

Passed all tests! ✓

Đúng

Marks for this submission: 1,00/1,00.



Câu hỏi 2

Đúng

Đạt điểm 1,00 trên 1,00

Implement methods **get**, **set**, **empty**, **indexOf**, **contains** in template class **DLinkedList** (which implements **List ADT**) representing the [singly linked list](#) with type T with the initialized frame. The description of each method is given in the code.

```
template <class T>
class DLinkedList {
public:
    class Node; // Forward declaration
protected:
    Node* head;
    Node* tail;
    int count;
public:
    DLinkedList();
    ~DLinkedList();
    void add(const T &e);
    void add(int index, const T &e);
    int size();
    bool empty();
    T get(int index);
    void set(int index, const T &e);
    int indexOf(const T &item);
    bool contains(const T &item);
public:
    class Node
    {
    private:
        T data;
        Node *next;
        Node *previous;
        friend class DLinkedList<T>;

    public:
        Node()
        {
            this->previous = NULL;
            this->next = NULL;
        }

        Node(const T &data)
        {
            this->data = data;
            this->previous = NULL;
            this->next = NULL;
        }
    };
};
```

In this exercise, we have include `<iostream>`, `<string>`, `<sstream>` and using namespace std.

For example:



Test	Result
<pre>DLinkedList<int> list; int size = 10; for(int idx=0; idx < size; idx++){ list.add(idx); } for(int idx=0; idx < size; idx++){ cout << list.get(idx) << " "; }</pre>	0 1 2 3 4 5 6 7 8 9
<pre>DLinkedList<int> list; int size = 10; int value[] = {2,5,6,3,67,332,43,1,0,9}; for(int idx=0; idx < size; idx++){ list.add(idx); } for(int idx=0; idx < size; idx++){ list.set(idx, value[idx]); } cout << list.toString();</pre>	[2,5,6,3,67,332,43,1,0,9]

Answer: (penalty regime: 0, 0, 0, 5, 10 %)

Reset answer

```

1  template<class T>
2  T DLinkedList<T>::get(int index) {
3      /* Give the data of the element at given index in the list. */
4      Node * tmp =head;
5      for (int i=0;i<index;i++){
6          tmp=tmp->next;
7      }
8      return tmp->data;
9
10
11 }
12
13 template <class T>
14 void DLinkedList<T>::set(int index, const T& e) {
15     /* Assign new value for element at given index in the list */
16     Node * tmp =head;
17     for (int i=0;i<index;i++){
18         tmp=tmp->next;
19     }
20     tmp->data=e;
21 }
22
23 template<class T>
24 bool DLinkedList<T>::empty() {
25     /* Check if the list is empty or not. */
26     return ( this->count ==0);
27 }
28
29
30 template<class T>
31 int DLinkedList<T>::indexOf(const T& item) {
32     /* Return the first index wheter item appears in list, otherwise return -1 */
33     Node * tmp =head;
34     for (int i=0;i<this->count;i++){
35         if (tmp->data==item ) return i;
36         tmp=tmp->next;
37     }
38     return -1;
39 }
40
41
42 template<class T>
43 bool DLinkedList<T>::contains(const T& item) {
44     /* Check if the item exists in the list */

```

```
44  /* Check if item appears in the list */
45  Node * tmp =head;
46  for (int i=0;i<this->count;i++){
47      if (tmp->data==item ) return true;
48      tmp=tmp->next;
49  }
50  return false;
51
52 }
53
54
```

	Test	Expected	Got	
✓	<pre>DLinkedList<int> list; int size = 10; for(int idx=0; idx < size; idx++){ list.add(idx); } for(int idx=0; idx < size; idx++){ cout << list.get(idx) << " "; }</pre>	0 1 2 3 4 5 6 7 8 9	0 1 2 3 4 5 6 7 8 9	✓
✓	<pre>DLinkedList<int> list; int size = 10; int value[] = {2,5,6,3,67,332,43,1,0,9}; for(int idx=0; idx < size; idx++){ list.add(idx); } for(int idx=0; idx < size; idx++){ list.set(idx, value[idx]); } cout << list.toString();</pre>	[2,5,6,3,67,332,43,1,0,9]	[2,5,6,3,67,332,43,1,0,9]	✓

Passed all tests! ✓

Đúng

Marks for this submission: 1,00/1,00.



Câu hỏi 3

Đúng

Đạt điểm 1,00 trên 1,00

Implement Iterator class in class DLinkedList.

Note: Iterator is a concept of repetitive elements on sequence structures. Iterator is implemented in class vector, list in STL container in C++ (<https://www.geeksforgeeks.org/iterators-c-stl/>). Your task is to implement the simple same class with iterator in C++ STL container.



```

template <class T>
class DLinkedList
{
public:
    class Iterator; //forward declaration
    class Node;     //forward declaration
protected:
    Node *head;
    Node *tail;
    int count;
public:
    DLinkedList() : head(NULL), tail(NULL), count(0){};
    ~DLinkedList();
    void add(const T &e);
    void add(int index, const T &e);
    T removeAt(int index);
    bool removeItem(const T &item);
    bool empty();
    int size();
    void clear();
    T get(int index);
    void set(int index, const T &e);
    int indexOf(const T &item);
    bool contains(const T &item);
    string toString();
    Iterator begin()
    {
        return Iterator(this, true);
    }
    Iterator end()
    {
        return Iterator(this, false);
    }
public:
    class Node
    {
    private:
        T data;
        Node *next;
        Node *previous;
        friend class DLinkedList<T>;

        Iterator begin()
        {
            return Iterator(this, true);
        }
        Iterator end()
        {
            return Iterator(this, false);
        }

    public:
        Node()
        {
            this->previous = NULL;
            this->next = NULL;
        }

        Node(const T &data)
        {
            this->data = data;
            this->previous = NULL;
            this->next = NULL;
        }
    };
};

```



```
class Iterator
{
private:
    DLinkedList<T> *pList;
    Node *current;
    int index; // is the index of current in pList
public:
    Iterator(DLinkedList<T> *pList, bool begin);
    Iterator &operator=(const Iterator &iterator);
    void set(const T &e);
    T &operator*();
    bool operator!=(const Iterator &iterator);
    void remove();

    // Prefix ++ overload
    Iterator &operator++();

    // Postfix ++ overload
    Iterator operator++(int);
};
```

Please read example carefully to see how we use the iterator.

For example:

Test	Result
DLinkedList<int> list; int size = 10; for(int idx=0; idx < size; idx++){ list.add(idx); } DLinkedList<int>::Iterator it = list.begin(); for(; it != list.end(); it++) { cout << *it << " "; }	0 1 2 3 4 5 6 7 8 9
DLinkedList<int> list; int size = 10; for (int idx = 0; idx < size; idx++) { list.add(idx); } DLinkedList<int>::Iterator it = list.begin(); while (it != list.end()) { it.remove(); it++; } cout << list.toString();	[]
DLinkedList<int> list; int size = 10; for (int idx = 0; idx < size; idx++) { list.add(idx); } DLinkedList<int>::Iterator it = list.begin(); for(; it != list.end(); it++) { it.remove(); } cout << list.toString();	[]

Answer: (penalty regime: 0, 0, 0, 5, 10 %)

Reset answer

```

1  template <class T>
2  DLinkedList<T>::Iterator::Iterator(DLinkedList<T> *pList, bool begin) {
3      this->pList = pList;
4      if (begin) {
5          current = pList->head;
6          if(pList != nullptr) this->index = 0;
7          else this->index = -1;
8      } else {
9          current = nullptr;
10         this->index = pList ? pList->size() : -1;
11     }
12 }
13
14 template <class T>
15 typename DLinkedList<T>::Iterator& DLinkedList<T>::Iterator::operator=(const DLinkedList<T>::Iterator &iterator)
16 {
17     if (this != &iterator) { // Kiểm tra nếu không tự gán
18         this->current = iterator.current;
19         this->index = iterator.index;
20         this->pList = iterator.pList;
21     }
22     return *this;
23 }
24
25 template <class T>
26 void DLinkedList<T>::Iterator::set(const T &e)
27 {
28     if (current) {
29         current->data = e;
30     }
31 }
32
33 template <class T>
34 T& DLinkedList<T>::Iterator::operator*()
35 {
36     return current->data;
37 }
38
39 template <class T>
40 void DLinkedList<T>::Iterator::remove()
41 {
42     if (current == nullptr) {
43         throw std::out_of_range("Segmentation fault!");
44     } else if (index == 0) {
45         pList->removeAt(0);
46         current = nullptr;
47         index = -1;
48     } else {
49         current = current->previous;
50         pList->removeAt(index);
51         index--;
52     }
53 }
54
55 template <class T>
56 bool DLinkedList<T>::Iterator::operator!=(const typename DLinkedList<T>::Iterator &iterator)
57 {
58     return this->current != iterator.current || this->index != iterator.index;
59 }
60
61 template <class T>
62 typename DLinkedList<T>::Iterator& DLinkedList<T>::Iterator::operator++()
63 {
64     if(this->index == this->pList->count) throw std::out_of_range("Segmentation fault!");
65     if(this->index == -1) {
66         this->current = this->pList->head;
67         this->index = 0;
68     } else {
69         this->current = this->current->next;
70         this->index++;
71     }
72 }

```

```

69         this->current = this->current->next;
70         this->index += 1;
71     }
72     return *this;
73 }
74
75 // Định nghĩa operator++(int) (postfix increment)
76 template <class T>
77 typename DLinkedList<T>::Iterator DLinkedList<T>::Iterator::operator++(int)
78 {
79     Iterator temp = *this;
80     ++(*this); // Sử dụng prefix increment để tăng giá trị
81     return temp;
82 }

```

	Test	Expected	Got	
✓	<pre> DLinkedList<int> list; int size = 10; for(int idx=0; idx < size; idx++){ list.add(idx); } DLinkedList<int>::Iterator it = list.begin(); for(; it != list.end(); it++) { cout << *it << " "; } </pre>	0 1 2 3 4 5 6 7 8 9	0 1 2 3 4 5 6 7 8 9	✓
✓	<pre> DLinkedList<int> list; int size = 10; for (int idx = 0; idx < size; idx++) { list.add(idx); } DLinkedList<int>::Iterator it = list.begin(); while (it != list.end()) { it.remove(); it++; } cout << list.toString(); </pre>	[]	[]	✓
✓	<pre> DLinkedList<int> list; int size = 10; for (int idx = 0; idx < size; idx++) { list.add(idx); } DLinkedList<int>::Iterator it = list.begin(); for(; it != list.end(); it++) { it.remove(); } cout << list.toString(); </pre>	[]	[]	✓

Passed all tests! ✓

Đúng

Marks for this submission: 1,00/1,00.



Câu hỏi 4

Đúng

Đạt điểm 1,00 trên 1,00

Implement methods **removeAt**, **removeItem**, **clear** in template class **SLinkedList** (which implements **List ADT**) representing the [singly linked list](#) with type T with the initialized frame. The description of each method is given in the code.

```
template <class T>
class DLinkedList {
public:
    class Node; // Forward declaration
protected:
    Node* head;
    Node* tail;
    int count;
public:
    DLinkedList();
    ~DLinkedList();
    void add(const T &e);
    void add(int index, const T &e);
    int size();
    bool empty();
    T get(int index);
    void set(int index, const T &e);
    int indexOf(const T &item);
    bool contains(const T &item);
    T removeAt(int index);
    bool removeItem(const T &item);
    void clear();
public:
    class Node
    {
    private:
        T data;
        Node *next;
        Node *previous;
        friend class DLinkedList<T>;

    public:
        Node()
        {
            this->previous = NULL;
            this->next = NULL;
        }

        Node(const T &data)
        {
            this->data = data;
            this->previous = NULL;
            this->next = NULL;
        }
    };
};
```

In this exercise, we have include <iostream>, <string>, <sstream> and using namespace std.

For example:



Test	Result
<pre>DLinkedList<int> list; int size = 10; int value[] = {2,5,6,3,67,332,43,1,0,9}; for(int idx=0; idx < size; idx++){ list.add(value[idx]); } list.removeAt(0); cout << list.toString();</pre>	[5,6,3,67,332,43,1,0,9]

Answer: (penalty regime: 0 %)

Reset answer

```

1  template <class T>
2  T DLinkedList<T>::removeAt(int index) {
3      if (index < 0 || index >= count) {
4          throw out_of_range("Index is out of range");
5      }
6
7      Node* current = head;
8      T removedData;
9
10     if (index == 0) { // Removing the head
11         removedData = head->data;
12         head = head->next;
13         if (head) {
14             head->previous = nullptr;
15         } else {
16             tail = nullptr; // List is now empty
17         }
18     } else {
19         for (int i = 0; i < index; ++i) {
20             current = current->next;
21         }
22         removedData = current->data;
23         current->previous->next = current->next;
24         if (current->next) {
25             current->next->previous = current->previous;
26         } else {
27             tail = current->previous; // Removing the tail
28         }
29     }
30
31     delete current;
32     count--;
33     return removedData;
34 }
35
36 template <class T>
37 bool DLinkedList<T>::removeItem(const T& item) {
38     Node* current = head;
39
40     while (current != nullptr) {
41         if (current->data == item) {
42             if (current == head) { // Removing the head
43                 head = head->next;
44                 if (head) {
45                     head->previous = nullptr;
46                 } else {
47                     tail = nullptr; // List is now empty
48                 }
49             } else {
50                 current->previous->next = current->next;
51                 if (current->next) {
52                     current->next->previous = current->previous;
53                 } else {
54                     tail = current->previous; // Removing the tail

```

```

55         }
56     }
57     delete current;
58     count--;
59     return true;
60 }
61     current = current->next;
62 }
63
64     return false;
65 }
66
67 template<class T>
68 void DLinkedList<T>::clear() {
69     Node* current = head;
70     while (current != nullptr) {
71         Node* nextNode = current->next;
72         delete current;
73         current = nextNode;
74     }
75     head = tail = nullptr;
76     count = 0;
77 }
78

```

	Test	Expected	Got	
✓	DLinkedList<int> list; int size = 10; int value[] = {2,5,6,3,67,332,43,1,0,9}; for(int idx=0; idx < size; idx++){ list.add(value[idx]); } list.removeAt(0); cout << list.toString();	[5,6,3,67,332,43,1,0,9]	[5,6,3,67,332,43,1,0,9]	✓

Passed all tests! ✓

Đúng

Marks for this submission: 1,00/1,00.



Câu hỏi 5

Đúng

Đạt điểm 1,00 trên 1,00

In this exercise, we will use [Standard Template Library List](#) (click open in other tab to show more) to implement a Data Log. This is a simple implementation in applications using undo and redo. For example in Microsoft Word, you must have nodes to store states when Ctrl Z or Ctrl Shift Z to go back or forward.

DataLog has a doubly linked list to store the states of data (an integer) and iterator to mark the current state. Each state is stored in a node, the transition of states is depicted in the figure below.

Your task in this exercise is implement functions marked with `/* * TODO */`.

```
class DataLog
{
private:
    list<int> logList;
    list<int>::iterator currentState;

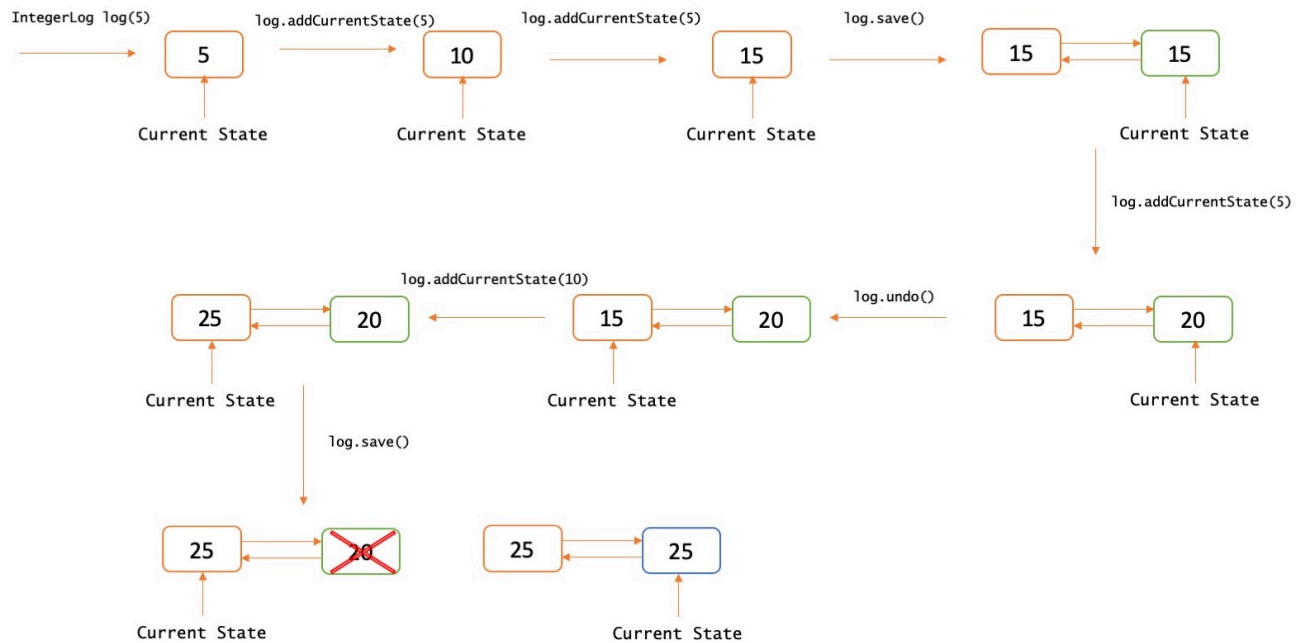
public:
    DataLog();
    DataLog(const int &data);
    void addCurrentState(int number);
    void subtractCurrentState(int number);
    void save();
    void undo();
    void redo();

    int getCurrentStateData()
    {
        return *currentState;
    }

    void printLog()
    {
        for (auto i = logList.begin(); i != logList.end(); i++) {
            if(i == currentState) cout << "Current state: ";
            cout << "[ " << *i << " ] => ";
        }
        cout << "END_LOG";
    }
};
```

Note: Normally, when we say a List, we talk about doubly linked list. For implementing a [singly linked list](#), we use forward list. We have include `<iostream>` `<list>` and using namespace std;





For example:

Test	Result
<pre>DataLog log(10); log.save(); log.addCurrentState(15); log.save(); log.addCurrentState(15); log.undo(); log.printLog();</pre>	<pre>[10] => Current state: [25] => [40] => END_LOG</pre>
<pre>DataLog log(10); log.save(); log.addCurrentState(15); log.save(); log.addCurrentState(15); log.save(); log.subtractCurrentState(5); log.printLog();</pre>	<pre>[10] => [25] => [40] => Current state: [35] => END_LOG</pre>

Answer: (penalty regime: 0, 0, 0, 5, 10 %)

Reset answer

```

1 DataLog:: DataLog() {
2     // Default constructor
3     logList.push_back(0); // Initialize with default value 0
4     currentState = logList.begin();
5 }
6
7 DataLog:: DataLog(const int &data) {
8     // Constructor with initial state
9     logList.push_back(data);
10    currentState = logList.begin();
11 }
12
13 void DataLog:: addCurrentState(int number) {
14     *currentState += number;
15 }
16

```

```

17 void DataLog:: subtractCurrentState(int number) {
18     *currentState -= number;
19 }
20
21 void DataLog:: save() {
22     // Erase all states forward of the current state to ensure linear history
23     logList.erase(next(currentState), logList.end());
24     // Save the current state and move the iterator to the new current state
25     logList.push_back(*currentState);
26     currentState = prev(logList.end());
27 }
28
29 void DataLog:: undo() {
30     if (currentState != logList.begin()) {
31         --currentState;
32     }
33 }
34
35 void DataLog:: redo() {
36     if (next(currentState) != logList.end()) {
37         ++currentState;
38     }
39 }
40
41
42

```

	Test	Expected	Got	
✓	DataLog log(10); log.save(); log.addCurrentState(15); log.save(); log.addCurrentState(15); log.undo(); log.printLog();	[10] => Current state: [25] => [40] => END_LOG	[10] => Current state: [25] => [40] => END_LOG	✓
✓	DataLog log(10); log.save(); log.addCurrentState(15); log.save(); log.addCurrentState(15); log.save(); log.subtractCurrentState(5); log.printLog();	[10] => [25] => [40] => Current state: [35] => END_LOG	[10] => [25] => [40] => Current state: [35] => END_LOG	✓

Passed all tests! ✓

Đúng

Marks for this submission: 1,00/1,00.



Câu hỏi 6

Đúng

Đạt điểm 1,00 trên 1,00

Given the head of a doubly linked list, two positive integer a and b where $a \leq b$. Reverse the nodes of the list from position a to position b and return the reversed list

Note: the position of the first node is 1. It is guaranteed that a and b are valid positions. You MUST NOT change the val attribute in each node.

```
struct ListNode {
    int val;
    ListNode *left;
    ListNode *right;
    ListNode(int x = 0, ListNode *l = nullptr, ListNode* r = nullptr) : val(x), left(l), right(r) {}
};
```

Constraint:

$1 \leq \text{list.length} \leq 10^5$

$0 \leq \text{node.val} \leq 5000$

$1 \leq \text{left} \leq \text{right} \leq \text{list.length}$

Example 1:

Input: list = {3, 4, 5, 6, 7}, a = 2, b = 4

Output: 3 6 5 4 7

Example 2:

Input: list = {8, 9, 10}, a = 1, b = 3

Output: 10 9 8

For example:

Test	Input	Result
<pre>int size; cin >> size; int* list = new int[size]; for(int i = 0; i < size; i++) { cin >> list[i]; } int a, b; cin >> a >> b; unordered_map<ListNode*, int> nodeValue; ListNode* head = init(list, size, nodeValue); ListNode* reversed = reverse(head, a, b); try { printList(reversed, nodeValue); } catch(char const* err) { cout << err << '\n'; } freeMem(head); delete[] list;</pre>	<pre>5 3 4 5 6 7 2 4</pre>	<pre>3 6 5 4 7</pre>



Test	Input	Result
<pre> int size; cin >> size; int* list = new int[size]; for(int i = 0; i < size; i++) { cin >> list[i]; } int a, b; cin >> a >> b; unordered_map<ListNode*, int> nodeValue; ListNode* head = init(list, size, nodeValue); ListNode* reversed = reverse(head, a, b); try { printList(reversed, nodeValue); } catch(char const* err) { cout << err << '\n'; } freeMem(head); delete[] list; </pre>	<pre> 3 8 9 10 1 3 </pre>	<pre> 10 9 8 </pre>

Answer: (penalty regime: 0 %)

Reset answer

```

1 ▾ ListNode* reverse(ListNode* head, int a, int b) {
2     if (!head || a == b) return head;
3
4     // Tìm nút tại vị trí a và b
5     ListNode* startPrev = nullptr;
6     ListNode* start = head;
7 ▾   for (int i = 1; i < a; ++i) {
8         startPrev = start;
9         start = start->right;
10    }
11
12    ListNode* end = start;
13 ▾   for (int i = a; i < b; ++i) {
14         end = end->right;
15    }
16    ListNode* endNext = end->right;
17
18    // Đảo ngược đoạn từ start đến end
19    ListNode* prev = endNext;
20    ListNode* current = start;
21 ▾   while (current != endNext) {
22         ListNode* next = current->right;
23         current->right = prev;
24         current->left = next;
25         prev = current;
26         current = next;
27    }
28
29    // Kết nối lại với phần trước a và sau b
30 ▾   if (startPrev) {
31         startPrev->right = end;
32 ▾   } else {
33         head = end;
34    }
35    start->right = endNext;
36 ▾   if (endNext) {
37         endNext->left = start;
38    }
39
40    return head;
41 }
42
43

```

	Test	Input	Expected	Got	
✓	<pre> int size; cin >> size; int* list = new int[size]; for(int i = 0; i < size; i++) { cin >> list[i]; } int a, b; cin >> a >> b; unordered_map<ListNode*, int> nodeValue; ListNode* head = init(list, size, nodeValue); ListNode* reversed = reverse(head, a, b); try { printList(reversed, nodeValue); } catch(char const* err) { cout << err << '\n'; } freeMem(head); delete[] list; </pre>	<pre> 5 3 4 5 6 7 2 4 </pre>	3 6 5 4 7	3 6 5 4 7	✓
✓	<pre> int size; cin >> size; int* list = new int[size]; for(int i = 0; i < size; i++) { cin >> list[i]; } int a, b; cin >> a >> b; unordered_map<ListNode*, int> nodeValue; ListNode* head = init(list, size, nodeValue); ListNode* reversed = reverse(head, a, b); try { printList(reversed, nodeValue); } catch(char const* err) { cout << err << '\n'; } freeMem(head); delete[] list; </pre>	<pre> 3 8 9 10 1 3 </pre>	10 9 8	10 9 8	✓

Passed all tests! ✓

Đúng

Marks for this submission: 1,00/1,00.

